UNIVERSIDAD MAYOR
para espíritus emprendedores



Robin

Proyecto - Arquitectura de desarrollo móvil y web

Nombre:
Lucas Calquín
Nicolás Escobar

Docente:
Carlos Muñoz

Sección:
INGT1003-001

Tabla de contenidos

1 Introducción 4

2 Trazabilidad con el Hito 1 5

2.1 Definición del Hito 1 5

2.2 Sección de cambios 6

3 Diseño de API REST 8

3.1 Criterios de diseño 8

3.2 Criterios de versionado 9

3.3 Tabla de recursos 9

3.4 Códigos de estado y formato de errores 11

3.5 Ejemplos de carga útil 12

4 Separación de responsabilidades y capas 14

4.1 Arquitectura en capas 14

4.2 Estructura de carpetas 15

4.3 Reglas de capas 16

4.4 Recorrido de peticiones 17

5 Persistencia y manejo de credenciales 20

5.1 Modelo de datos 20

5.2 Scripts y credenciales 21

6 Conclusión 24

7 Referencias técnicas 25

8 Anexo: uso de IA 27

8.1 Herramientas utilizadas 27

8.2 Uso por fase del proyecto 27

8.3 Verificación de resultados 28

8.4 Uso de la API base previa 29

Índice de figuras

Figura 1. Ciclo de vida de un ítem de la wishlist y transiciones de estado. 7
Figura 2. Esquema de versionado: ruta mayor y versionado semántico de la release. 9
Figura 3. Arquitectura en capas del backend de Robin: flujo de la petición (gris) y de la respuesta (verde). 15
Figura 4. Recorrido de `POST /api/v1/reservations` por las capas; en rojo, el código de error que puede originar cada una. 17
Figura 5. Modelo de datos con enfoque mixto: necesidades embebidas en la publicación y colecciones referenciadas mediante `ObjectId`. 20

Índice de tablas

Tabla 1. Cambios respecto de la propuesta del Hito 1 6
Tabla 2. Recursos de usuarios 9
Tabla 3. Recursos de publicaciones 10
Tabla 4. Recursos de reservas 11
Tabla 5. Códigos de estado HTTP utilizados por la API (Fielding et al., 2022) 11
Tabla 6. Qué puede y qué no puede hacer cada capa 16
Tabla 7. Recorrido de la petición de reserva a través de las capas 17
Tabla 8. Herramientas utilizadas como apoyo durante el proyecto 27
Tabla 9. Participación de la IA y del equipo por fase 28
Tabla 10. Elementos reutilizados, reescritos y nuevos 29

1 Introducción

Este documento describe el diseño técnico del backend del proyecto **“Robin”**, una plataforma web de wishlist solidaria. En ella, personas naturales publican sus necesidades concretas y reciben ayuda en bienes no monetarios de donantes, con verificación previa de cada historia mediante un rol de moderador y confirmación de la entrega.

Este avance corresponde al **Hito 2** del proyecto semestral de Arquitectura de Desarrollo Móvil y Web y continúa la propuesta del Hito 1, donde se definieron el problema, la propuesta de valor, el stack tecnológico, la arquitectura de información y el modelado de datos.

El objetivo de este Hito es la construcción y documentación del lado del servidor: una API REST con persistencia real de datos, organizada en capas con responsabilidades separadas, con credenciales fuera del repositorio y con decisiones de arquitectura justificadas. El alcance se limita al back end; el frontend en React (React, s.f.) se integra en el Hito 3.

19	3	5	6
endpoints REST	roles de usuario	entidades de datos	capas del backend

2 Trazabilidad con el Hito 1

2.1. Definición del Hito 1

El Hito 1 estableció las bases conceptuales y técnicas sobre las que se construye el backend del proyecto. Los elementos que condicionan directamente el trabajo del Hito 2 son los siguientes:

1. **Problema y propuesta de valor:** Se identificó que la ayuda solidaria está dominada por intermediarios institucionales y que las personas naturales no pueden gestionar sus propias solicitudes. En los casos cuando una persona natural puede lanzar su propia campaña de donación, estas se basan en donaciones monetarias, lo que puede exponer a los donantes a estafas mediante la realización de campañas falsas. Robin propone una wishlist de bienes, donde la unidad de valor es el ítem a donar y no el dinero, con un ciclo de vida disponible $\rightarrow$ reservado $\rightarrow$ entregado, verificación previa de cada publicación por un moderador y confirmación fotográfica de cada entrega.
2. **Stack y lenguaje**: Se definió el stack MERN (MongoDB, s.f.-d; Express, s.f.-b; React, s.f.; Node.js, s.f.) y se eligió TypeScript como lenguaje principal, aceptando una curva de aprendizaje adicional a cambio de detección temprana de errores y tipado compartido entre capas (Microsoft, s.f.).
3. **Arquitectura**: Se propuso una arquitectura cliente-servidor de tres capas. El servidor es una API REST en Node.js y Express (Express, s.f.-c; Express, s.f.-d) organizada internamente en routes, controllers, services y middlewares (autenticación JSON Web Token (Jones et al., 2015), validación de payloads y manejo centralizado de errores). La persistencia usa MongoDB a través de la librería Mongoose (Mongoose, s.f.-b).
4. **Arquitectura de información**: Se definieron tres roles (beneficiario, donante y moderador), el diagrama de navegación y un feed que muestra solo publicaciones, dejando las necesidades y la acción de reservar en el detalle de cada publicación.
5. **Modelo de datos**: El modelo lógico contempla cinco entidades: Usuario, Publicación, Necesidad, Reserva y Notificación. El modelo físico (JSON) adopta un enfoque mixto: las necesidades se embeben como subdocumentos de la publicación, y usuarios, reservas y notificaciones son colecciones independientes referenciadas (MongoDB,

s.f.-c) mediante ObjectId.[1]

6. **Plan de trabajo:** La malla de actividades asignó al Hito 2 la definición de la interfaz de programación y el modelado de datos, la implementación de la autenticación y de los endpoints REST, la documentación técnica y el testeo de la API.

2.2. Sección de cambios

Esta sección declara qué se conserva y qué se modifica respecto de la propuesta del Hito 1, junto con el motivo y el impacto de cada cambio.

2.2.1. Elementos que se conservan

- El concepto de wishlist de bienes y su ciclo de vida disponible → reservado → entregado.
- Los tres roles de usuario (beneficiario, donante, moderador) y el flujo de moderación previa a la publicación.
- El stack Node.js, Express y MongoDB, con autenticación mediante JWT.[2]
- La separación entre routes, controllers, services y middlewares.
- El modelo de datos de cinco entidades y el enfoque mixto de embebido y referenciado.
- La decisión de que el feed exponga solo publicaciones de forma paginada y de que la reserva se realice desde el detalle.

2.2.2. Elementos que se modifican

Tabla 1. Cambios respecto de la propuesta del Hito 1

Qué cambia	Motivo	Impacto
Se agrega la capa repositories	Aislar la lógica de acceso a datos de la lógica de negocio para evitar el acoplamiento con la base de datos	Mayor mantenibilidad, facilidad para realizar pruebas unitarias y centralización de consultas críticas (reservas atómicas)

[1] Identificador único de 12 bytes que MongoDB asigna por defecto al campo `_id` de cada documento. (MongoDB, s.f.-b)

[2] JSON Web Token: estándar abierto (RFC 7519) para transmitir información firmada entre cliente y servidor. (Jones et al., 2015)

Qué cambia	Motivo	Impacto
Estado de la reserva: se define el estado cancelada	El prototipo permite cancelar una reserva, pero el Hito 1 no definía los estados de la reserva	Al cancelar, el ítem vuelve al estado disponible y puede ser reservado por otro donante
Contrato de la API: rutas orientadas a recursos con versionado `/api/v1`	El Hito 1 no definía nomenclatura de rutas ni criterio de versionado	Contrato estable y documentado, que permite integrar el frontend en el Hito 3 sin rediseñar el servidor
Confirmación fotográfica de entregas: se cambia por confirmación manual (donante y beneficiario confirman la entrega)	La subida de fotos exige manejo de archivos y plantea un riesgo de privacidad. Esa complejidad excede el mínimo del hito y desvía el tiempo disponible del flujo central de estados (disponible → reservado → entregado)	Se pierde evidencia verificable de la entrega, lo que debilita la garantía de trazabilidad. Se mitiga exigiendo la confirmación de ambas partes y registrando la fecha en `delivered_at`.
Plan de empresa y certificado de donación: Se elimina del proyecto y el ingreso queda solo en la comisión de afiliado	El certificado depende de una entidad donataria inscrita en el registro público de la Ley 21.440, cuya constitución y validación legal excede el alcance técnico del proyecto	Disminución de los ingresos dentro de la plataforma, pero con un modelo de ingresos más realista, ya que la fundación intermediaria solo se estableció como idea.



Figura 1. Ciclo de vida de un ítem de la wishlist y transiciones de estado.

3 Diseño de API REST

3.1. Criterios de diseño

La API REST (Fielding, 2000) de Robin se diseña bajo el principio *contract-first*:[3] la tabla de recursos de este capítulo es el acuerdo entre el servidor y la futura interfaz del Hito 3, y el código se ajusta a ella.

Criterios de diseño adoptados:

1. Semántica estándar de los verbos HTTP (Fielding et al., 2022): GET consulta sin efectos secundarios; POST crea; PATCH modifica parcialmente; PUT reemplaza; DELETE elimina.
2. Acciones de dominio modeladas como estado. En lugar de rutas como "/reserve", se modifica el estado del recurso (`PATCH /publication/{id}/status`) o se crea un recurso nuevo (`POST /reservation`).
3. JSON como formato de entrada y salida, salvo la subida de imágenes.
4. API sin estado y autenticación por JWT en la cabecera `Authorization` (Jones & Hardt, 2012). El middleware verifica el token y el rol (Express, s.f.-d). La verificación de propiedad del recurso (por ejemplo, que una publicación pertenezca al usuario) la hace la capa de servicios.
5. Paginación por parámetros de consulta y respuesta con metadatos de paginación.
6. El código de estado HTTP indica el resultado (Fielding et al., 2022). Se repite en el cuerpo con campos como `status: "success"`.
7. Respuestas sin datos sensibles (no se devuelven contraseñas) y errores sin detalles internos.
8. Reglas de negocio atómicas. La reserva de un ítem a donar se resuelve con una actualización condicionada al estado disponible, para evitar reservas duplicadas (MongoDB, s.f.-a).

[3] Enfoque en el que el contrato de la API se define y acuerda antes de escribir el código que lo implementa.

3.2. Criterios de versionado

- **Versión en la ruta:** Toda la API se expone bajo `/api/v1`. Se eligió porque es explícito, simple de probar con cualquier cliente y de documentar; se descartaron la cabecera personalizada y el parámetro de consulta por ser menos visibles.
- **Esquema** `/v1` identifica la versión mayor del contrato, y el número de release sigue versionado semántico (MAJOR.MINOR.PATCH) (Preston-Werner, s.f.). La entrega de este hito corresponde a la release 1.0.0.
- **Cambios compatibles:** nuevos endpoints, campos opcionales, nuevos valores aceptados se publican como versión menor (1.1.0) sin cambiar la ruta.
- **Cambios incompatibles:** renombrar o eliminar un campo, cambiar un código de estado o la estructura de un error, requieren una nueva versión mayor (`/api/v2`).

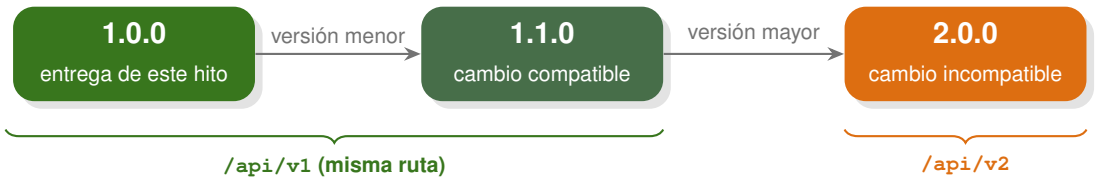


Figura 2. Esquema de versionado: ruta mayor y versionado semántico de la release.

3.3. Tabla de recursos

Todas las rutas parten de `/api/v1`. Acceso: Público, Autenticado, o el rol requerido (beneficiario, donante, administrador). Códigos de error comunes a todo endpoint autenticado: 401 (sin token, inválido o expirado) y 500; solo se listan los específicos.

Usuarios

Tabla 2. Recursos de usuarios

Verbo	Ruta	Cuerpo (JSON)	Acceso	Éxito	Errores
POST	`/users/register` *registrar usuario*	name, surname, nick, email, password, role	Público	201	400, 409
POST	`/users/login` *iniciar sesión*	email, password	Público	200	400, 401
GET	`/users/profile/:id` *ver perfil de usuario*	–	JWT	200	400, 404

Verbo	Ruta	Cuerpo (JSON)	Acceso	Éxito	Errores
GET	`/users/list` *listar usuarios*	–	JWT	200	400
GET	`/users/list/:page` *listar usuarios paginado*	–	JWT	200	400
PATCH	`/users/update` *actualizar perfil*	name, surname, nick, email, password, role (campos opcionales)	JWT	200	400, 409
POST	`/users/upload` *subir avatar*	multipart/form-data (campo “file”)	JWT	200	400, 415
GET	`/users/avatar/:file` *obtener imagen de avatar*	–	Público	200	404

Publicaciones

Tabla 3. Recursos de publicaciones

Verbo	Ruta	Cuerpo (JSON)	Acceso	Éxito	Errores
POST	`/publications` *crear publicación*	text, necesidades[{name}]	JWT (Beneficiario)	201	400, 403
GET	`/publications` *ver publicaciones aprobadas*	–	JWT	200	400
GET	`/publications/:id` *detalle de publicación*	–	JWT	200	400, 404
PATCH	`/publications/:id` *editar publicación*	text, necesidades[{name}]	JWT (Dueño)	200	400, 403, 404
DELETE	`/publications/:id` *eliminar publicación*	–	JWT (Dueño)	204	403, 404, 409
PATCH	`/publications/:id/status` *verificar publicación*	status: aprobada o rechazada	JWT (Admin)	200	400, 403, 404, 409
PUT	`/publications/:id/image` *subir imagen de publicación*	multipart/form-data (campo “file”)	JWT (Dueño)	200	400, 403, 404, 415

Reservas

Tabla 4. Recursos de reservas

Verbo	Ruta	Cuerpo (JSON)	Acceso	Éxito	Errores
POST	`/reservations` *reservar un ítem*	publication_id, item_id	JWT (Donante)	201	400, 403, 404, 409
GET	`/reservations` *mis reservas*	–	JWT	200	400
GET	`/reservations/:id` *detalle de la reserva*	–	JWT (Dueño, donante o admin)	200	403, 404
PATCH	`/reservations/:id/status` *cambiar estado de la reserva*	status: cancelada o entregado	JWT (Dueño o donante)	200	400, 403, 404, 409

3.4. Códigos de estado y formato de errores

Códigos de estado utilizados

Tabla 5. Códigos de estado HTTP utilizados por la API (Fielding et al., 2022)

Código	Uso
200	Consulta o modificación exitosa (también lista vacía)
201	Recurso creado (usuario, publicación, reserva)
204	Eliminación exitosa, sin cuerpo
400	Cuerpo, parámetros o identificador mal formados
401	Falta el token, es inválido o expiró
403	Autenticado pero sin permiso (rol)
404	El recurso no existe
409	Conflicto de estado (necesidad/item ya reservado, correo repetido, transición no permitida)
415	Archivo de tipo no permitido
500	Falla inesperada del servidor

Formato de respuesta estándar

Éxito

```
{
  "status": "success",
  "statusCode": 200,
  "data": { ... },
  "meta": { "page": 1, "limit": 10, "total": 100, "pages": 10 }
}
```

Error

```
{
  "status": "error",
  "statusCode": 409,
  "error": { "code": "NEED_NOT_AVAILABLE", "message": "...", "details": [] }
}
```

3.5. Ejemplos de carga útil

Petición: POST /api/v1/reservations

```
{
  "publication_id": "6ae5b687461f2637882e28ca",
  "item_id": "6ae7b8b9f598b91e5c8cad76"
}
```

Respuesta: 201 Created

```
{
  "status": "success",
  "statusCode": 201,
  "data": {
    "_id": "6ae8fc710a6623739cc888ff",
    "publication_id": "6ae5b687461f2637882e28ca",
    "item_id": "6ae7b8b9f598b91e5c8cad76",
    "status": "reservado",
    "reserved_at": "2026-11-02T21:11:13.743Z"
  }
}
```

Caso: Error de Permisos (Forbidden)

Petición: **DELETE** `/api/v1/publications/{id}` (siendo un usuario que no es el dueño).
Respuesta: **403 Forbidden**

Respuesta: 403 Forbidden

```
{
  "status": "error",
  "statusCode": 403,
  "error": {
    "code": "FORBIDDEN",
    "message": "No tienes permiso para eliminar esta publicación",
    "details": []
  }
}
```

4 Separación de responsabilidades y capas

4.1. Arquitectura en capas

El back end de Robin se organiza como un monolito en capas sobre Express (Express, s.f.-b). Cada petición HTTP atraviesa una cadena de capas con una única responsabilidad, y cada capa se comunica solo con la inmediatamente inferior. Se mantiene lo definido en el Hito 1 (routes, controllers, services y middlewares) y se agrega la capa repositories, según lo declarado en la sección 2.2.

Las capas se dividen en dos grupos. Las que conocen HTTP (routes, middlewares y controllers) trabajan con `req` y `res`. Las que no lo conocen (services, repositories y models) reciben y devuelven datos, por lo que pueden probarse y reutilizarse sin levantar el servidor. Además, hay componentes transversales usados por todas las capas: **config** (variables de entorno (Node.js, s.f.) y conexión a la base), **utils** (sobre de respuesta, errores de aplicación y generación del JWT) y el manejador central de errores (Express, s.f.-a).

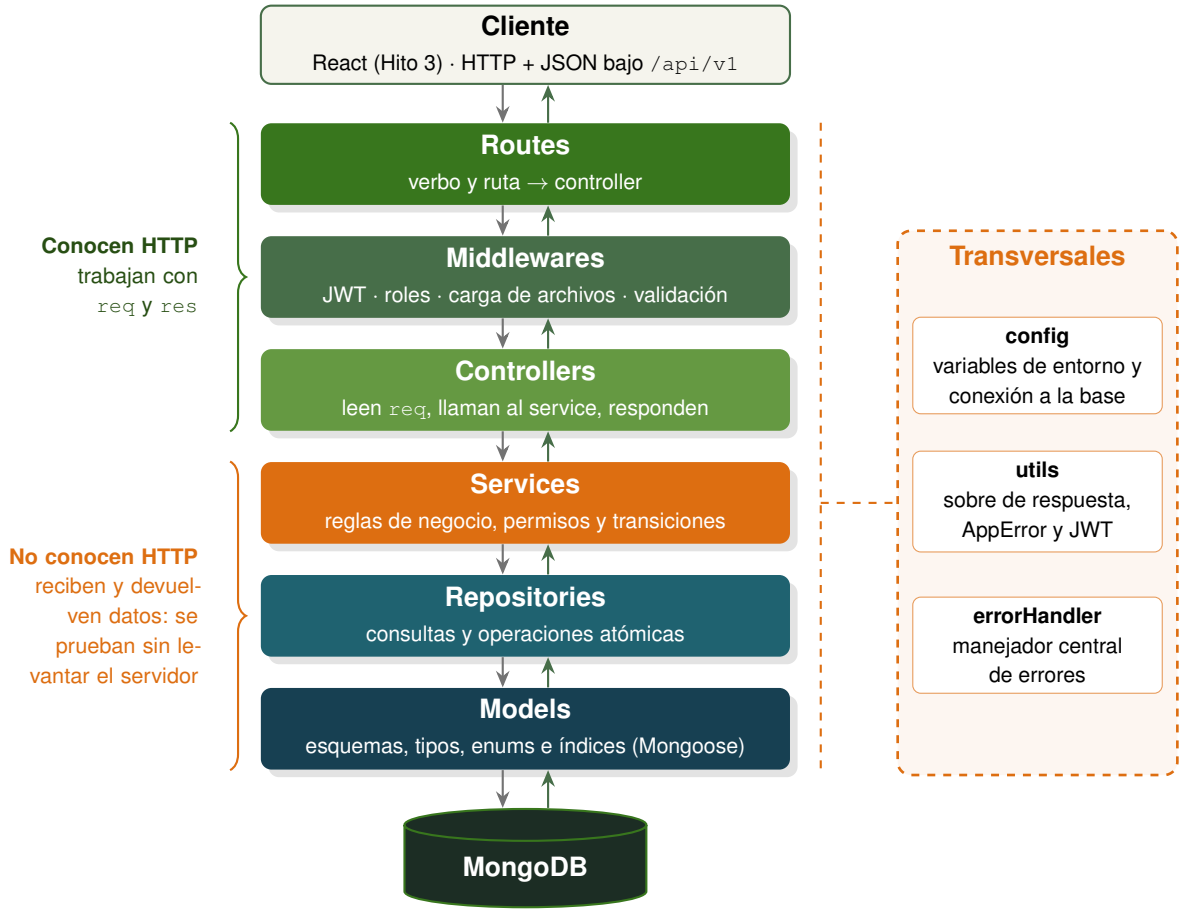


Figura 3. Arquitectura en capas del backend de Robin: flujo de la petición (gris) y de la respuesta (verde).

4.2. Estructura de carpetas

src/

```
+-- server.ts                # Arranque (Entry Point)
+-- app.ts                   # Configuración Global (Express, Middlewares Globales)
|
+-- routes/                  # ■ CAPA: ROUTES (Asocia Verbo/Ruta → Controller)
|   +-- user.ts              # Rutas de usuarios
|   +-- publication.ts       # Rutas de publicaciones
|   +-- reservation.ts       # Rutas de reservas
|
+-- middlewares/             # ■ CAPA: MIDDLEWARES (Validación de Token, Rol y Payload)
|   +-- auth.ts              # Verificación de JWT
|   +-- roles.ts             # Control de acceso por rol
|   +-- upload.ts            # Manejo de archivos (Multer)
|   +-- errorHandler.ts      # Manejador global de errores
```

```
|
+-- controllers/              # ■ CAPA: CONTROLLERS (HTTP → Servicio → Respuesta)
|   +-- user.ts               # Lógica de entrada/salida de Usuarios
|   +-- publication.ts        # Lógica de entrada/salida de Publicaciones
|   +-- reservation.ts        # Lógica de entrada/salida de Reservas
|
+-- services/                 # ■ CAPA: SERVICES (Lógica de Negocio y Permisos de Propiedad)
|   +-- auth.service.ts       # Lógica de autenticación
|   +-- user.service.ts       # Reglas de negocio de Usuarios
|   +-- publication.service.ts # Reglas de negocio de Publicaciones
|   +-- reservation.service.ts # Reglas de negocio de Reservas
|   +-- notification.service.ts # Reglas de negocio de Notificaciones
|   +-- jwt.ts                # Gestión de tokens (Lógica de servicio)
|
+-- repositories/             # ■ CAPA: REPOSITORIES (Consultas Atómicas y Persistencia)
|   +-- user.repository.ts          # Consultas MongoDB de Usuarios
|   +-- publication.repository.ts   # Consultas MongoDB de Publicaciones
|   +-- reservation.repository.ts   # Consultas MongoDB de Reservas
|
+-- models/                   # ■ CAPA: MODELS / MongoDB (Esquemas y Tipos)
|   +-- user.ts               # Esquema de Usuario
|   +-- publication.ts        # Esquema de Publicación
|   +-- reservation.ts        # Esquema de Reserva
|   +-- notification.ts       # Esquema de Notificación
|
+-- types/                    # TIPADO (Específico de TypeScript)
|   +-- domain.ts             # Tipos y Enums del dominio (Roles, Estados)
|   +-- express.d.ts          # Extensión de Request para incluir req.user
|
+-- utils/                    # Soporte: Utilidades transversales
|   +-- response.ts           # Constructor del sobre de respuesta
|   +-- AppError.ts           # Clase de errores de negocio
|
+-- validators/               # Soporte: Validaciones de entrada
|
+-- database/                 # Configuración de Base de Datos
|   +-- connection.ts         # Gestión de conexión a MongoDB
|
+-- helpers/                  # Soporte: Ayudantes generales
    +-- validate.ts           # Lógica auxiliar de validación
```

4.3. Reglas de capas

Tabla 6. Qué puede y qué no puede hacer cada capa

Capa	Puede	No puede
Route	Declarar endpoint y verbo; encadenar middlewares y controller	Contener lógica o consultas.
Middlewares	Verificar token, rol y forma de los datos	Aplicar reglas de negocio o consultar la base

Capa	Puede	No puede
Controllers	Leer `req`, llamar a un service y responder con `utils/response`	Importar repositories o models; decidir reglas; armar el sobre manualmente.
Services	Aplicar reglas, permisos y transiciones; llamar repositories/services; lanzar `AppError`	Conocer `req`, `res`, Express, Mongoose o models
Repositories	Ejecutar consultas y operaciones atómicas.	Contener reglas de negocio o conocer HTTP
Models	Definir esquema, tipos, enums e índices.	Contener lógica de negocio o consultas.

Regla de dependencia: Las dependencias van hacia abajo: routes → middlewares/controllers → services → repositories → models. Ninguna capa se salta a la siguiente ni depende de una superior.

4.4. Recorrido de peticiones

Ejemplo: Reservar un ítem de una publicación

A continuación se describe la travesía de una petición **POST** `/api/v1/reservations`.

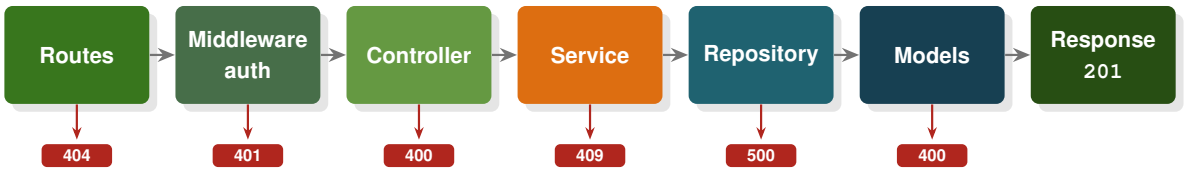


Figura 4. Recorrido de `POST /api/v1/reservations` por las capas; en rojo, el código de error que puede originar cada una.

Tabla 7. Recorrido de la petición de reserva a través de las capas

Capa	Archivo	Caso exitoso	Caso de error
Routes	`routes/reservation.t:`	Recibe la petición y verifica que el verbo sea POST. Delega al middleware de auth y luego al controlador.	**404 Not Found:** Si la ruta no existe.

Capa	Archivo	Caso exitoso	Caso de error
Middleware	`auth.ts`	Extrae el token Bearer, lo valida y adjunta el usuario al objeto `req.user`.	**401 Unauthorized:** Token inválido o ausente.
Controller	`reservation.ts`	Extrae `publication_id` e `item_id` del cuerpo. Llama al servicio de reservas.	**400 Bad Request:** Si faltan campos obligatorios (lanza `AppError`).
Service	`reservation.service.ts`	Valida que la publicación esté aprobada y el ítem disponible. Orquesta la reserva atómica.	**409 Conflict:** Si el ítem ya está reservado o la publicación fue rechazada.
Repository	`reservation.repository.ts`	Ejecuta el `findOneAndUpdate` (Mongoose, s.f.-a; MongoDB, s.f.-a) atómico en MongoDB y crea el documento de reserva.[4]	**500 Internal Error:** Fallo de conexión o error de base de datos.
Models	`reservation.ts`	Valida que los datos cumplan el esquema (ObjectId, Enums de estado) (Mongoose, s.f.-c).	**400 Validation Error:** Datos incompatibles con el esquema.
Response	`utils/response.ts`	El controlador recibe el resultado y usa `sendSuccess` para envolver la data en el sobre estándar.	—

1 · ROUTE

```
// 1. ROUTE: Definición del endpoint
// src/routes/reservation.ts
router.post("/", auth, ReservationController.create); // Enlace: Ruta -> Auth ->
  ↪ Controller
```

2 · CONTROLLER

```
// 2. CONTROLLER: Traducción HTTP -> Servicio
// src/controllers/reservation.ts
async create(req: Request, res: Response, next: NextFunction) {
  try {
```

[4] Una operación sobre un único documento en MongoDB es atómica: la condición de búsqueda y la modificación se aplican como un solo paso, sin que otra petición pueda intercalarse.

```
    const { publication_id, item_id } = req.body;
    // Llama al servicio pasando el ID del usuario autenticado (req.user)
    const reservation = await reservationService.createReservation(req.user!.id,
  ↪ publication_id, item_id);
    sendSuccess(res, reservation, 201); // Respuesta estandarizada
  } catch (error) {
    next(error); // Propaga el error al errorHandler
  }
}
```

3 · SERVICE

```
// 3. SERVICE: Reglas de negocio y validación de estado
// src/services/reservation.service.ts
async createReservation(donorId, pubId, itemId) {
  // Lógica: Verificar que el ítem esté "disponible" antes de intentar reservar
  // Lanza AppError si no se cumple la regla de negocio
  if (!isAvailable) throw new AppError("NEED_NOT_AVAILABLE", 409, "El ítem ya está
  ↪ reservado");
  return await reservationRepository.save(donorId, pubId, itemId);
}
```

4 · REPOSITORY

```
// 4. REPOSITORY: Operación atómica en MongoDB
// src/repositories/reservation.repository.ts
async save(donorId, pubId, itemId) {
  // Actualización condicionada: Solo marca como 'reservado' si sigue 'disponible'
  return await Publication.findOneAndUpdate(
    { _id: pubId, "necesidades": { $elemMatch: { _id: itemId, status: "disponible" } }
  ↪  },
    { $set: { "necesidades.$.status": "reservado" } }
  );
}
```

5 Persistencia y manejo de credenciales

5.1. Modelo de datos

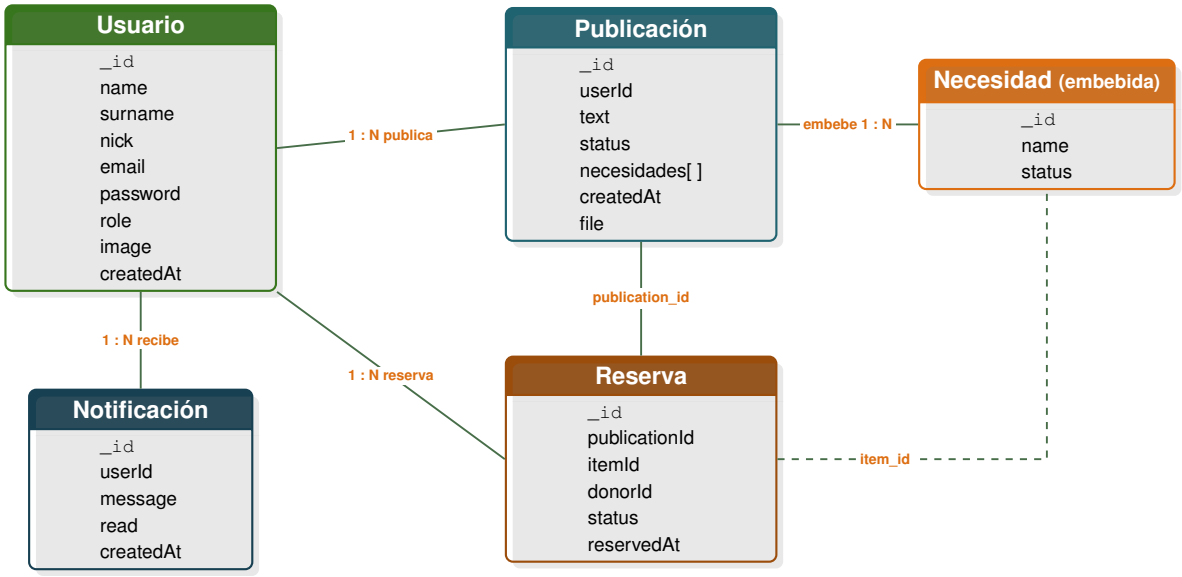


Figura 5. Modelo de datos con enfoque mixto: necesidades embebidas en la publicación y colecciones referenciadas mediante `ObjectId`.

Campos, tipos y restricciones

Users

```
{ _id: ObjectId, name: String, surname: String, nick: String, // unique
  email: String, password: String, // hash; nunca se devuelve
  role: String, // beneficiario | donante | admin
  image: String, address: String, phone: String, verified: Boolean,
  created_at: Date }
```

Publications

```
{ _id: ObjectId, user_id: ObjectId, text: String, file: String,
  status: String, // pendiente | aprobada | rechazada
  necesidades: [{ _id: ObjectId, name: String, status: String }],
    created_at: Date, file: String}
```

Reservations

```
{ _id: ObjectId, publication_id: ObjectId, item_id: ObjectId,
  donor_id: ObjectId, status: String, // reservado | entregado | cancelada
  reserved_at: Date, delivered_at: Date }
```

Notifications

```
{ _id: ObjectId, user_id: ObjectId,
  message: String, read: Boolean, created_at: Date }
```

Decisiones de diseño físico

- **Necesidades embebidas:** se leen junto con su publicación, su cantidad es acotada y se evita una consulta adicional al mostrar el detalle.
- **Reservations referenciadas:** tienen ciclo de vida propio y se consultan por donante y estado.
- **Users referenciados:** evita duplicar datos personales en cada publicación o reserva.

5.2. Scripts y credenciales

El despliegue y arranque del sistema se gestionan mediante scripts de Node.js definidos en el `package.json` (npm, s.f.). Para el desarrollo, se utiliza el comando `npm run dev`, que levanta el servidor en modo observador (watch mode), permitiendo que los cambios en el código se apliquen en tiempo real sin reiniciar el proceso manualmente (Node.js, s.f.).

La arquitectura de persistencia se divide en dos componentes con responsabilidades distintas:

1. **Gestión de Conexión (`src/database/connection.ts`):** Es la infraestructura que mantiene el enlace activo entre la aplicación y MongoDB. Se ejecuta en cada arranque del servidor, leyendo la `MONGODB\_URI` del archivo de entorno para establecer la comunicación persistente.

2. **Inicialización de Base de Datos (`db/init-db.js`):** Es un script de configuración que se ejecuta manualmente (vía `npm run db:init`) para preparar la base de datos desde cero. Sus funciones principales son:
   - Crear las colecciones necesarias.
   - Sincronizar los índices optimizados definidos en los modelos (ej. unicidad de email y nick) (MongoDB, s.f.-e).
   - Crear el usuario administrador inicial utilizando las credenciales definidas en el archivo de entorno.

A continuación se muestra el flujo de arranque recomendado y la configuración de entorno necesaria para desplegar el servicio:

1 · Preparación del entorno

```
cp .env.example .env       # Crear archivo de entorno local
npm install                 # Instalar dependencias del proyecto
```

2 · Preparación de la base de datos (solo la primera vez o tras reset)

```
npm run db:init             # Crear colecciones, indices y usuario admin
```

3 · Ejecución del servicio

```
npm run dev                 # Arranca servidor en modo desarrollo
```

.env.example (valores de ejemplo)

```
PORT=3000
MONGODB_URI=mongodb://localhost:27017/robin
JWT_SECRET=clave-secreta-aleatoria-larga
JWT_EXPIRES_IN=24h
CORS_ORIGIN=http://localhost:5173
ADMIN_EMAIL=admin@example.com
ADMIN_PASSWORD=clave-segura-admin
```

.gitignore

```
node_modules/
.env
uploads/*
!uploads/.gitkeep
run-project.bat
```

Prácticas adoptadas

1. **Exclusión de secretos:** el `.env` real no se versiona; proporcionando solo la plantilla `.env.example` (Wiggins, 2011).
2. **Falla temprana:** El sistema valida la existencia de las variables de entorno críticas al arrancar el servidor.
3. **Mínimo privilegio:** Se utiliza una URI de conexión restringida al entorno de la base de datos.
4. **Secretos por entorno:** desarrollo y despliegue usan valores distintos; `JWT\_SECRET` es largo y aleatorio (Wiggins, 2011).
5. **Rotación:** si una credencial se publica, se invalida, se reemplaza y se limpia el historial.

6 Conclusión

Este hito permitió construir y documentar el backend de Robin mediante una API REST con persistencia en MongoDB, organizada en capas con responsabilidades separadas y con credenciales protegidas fuera del repositorio. Se mantuvo el concepto definido en el Hito 1, basado en una wishlist de bienes con el ciclo de vida disponible → reservado → entregado, y se establecieron los cambios necesarios junto con sus motivos e impactos.

Las decisiones centrales fueron cuatro. **Primero,** adoptar una arquitectura de monolito en capas, adecuada para un proyecto semestral con un equipo pequeño y un despliegue sencillo. **Segundo,** incorporar la capa de repositorios (repositories), encargada de centralizar las consultas y mantener las reglas de negocio independientes de Mongoose. **Tercero,** definir un contrato de API orientado a recursos, versionado bajo `/api/v1` y con un formato de respuesta consistente. **Cuarto,** implementar un mecanismo de reserva atómica que impida que dos donantes reserven la misma necesidad.

1	2	3	4
Monolito en capas	Capa repositories	Contrato `/api/v1`	Reserva atómica

Asimismo, se reconocieron y documentaron las limitaciones del proyecto, tanto legales como técnicas, manteniendo un alcance que permita justificar y defender las decisiones adoptadas.

En el Hito 3, el backend servirá como base para integrar el frontend desarrollado en React y desplegar el sistema, culminando con la entrega de un producto funcional.

7 Referencias técnicas

- Express. (s. f.-a). *Error handling*. https://expressjs.com/en/guide/error-handling.html
- Express. (s. f.-b). *Express: Node.js web application framework*. https://expressjs.com/
- Express. (s. f.-c). *Routing*. https://expressjs.com/en/guide/routing.html
- Express. (s. f.-d). *Using middleware*. https://expressjs.com/en/guide/using-middleware.html
- Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm
- Fielding, R., Nottingham, M., & Reschke, J. (2022). *HTTP semantics* (RFC 9110). RFC Editor. https://www.rfc-editor.org/info/rfc9110
- Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). RFC Editor. https://www.rfc-editor.org/info/rfc7519
- Jones, M., & Hardt, D. (2012). *The OAuth 2.0 authorization framework: Bearer token usage* (RFC 6750). RFC Editor. https://www.rfc-editor.org/info/rfc6750
- Microsoft. (s. f.). *TypeScript documentation*. https://www.typescriptlang.org/docs/
- MongoDB. (s. f.-a). *Atomicity and transactions*. MongoDB Manual. https://www.mongodb.com/docs/manual/c operations-atomicity/
- MongoDB. (s. f.-b). *BSON types*. MongoDB Manual. https://www.mongodb.com/docs/manual/reference/bson types/
- MongoDB. (s. f.-c). *Embedded data versus references*. MongoDB Manual. https://www.mongodb.com/docs/m modeling/concepts/embedding-vs-references/
- MongoDB. (s. f.-d). *MongoDB manual*. https://www.mongodb.com/docs/manual/
- MongoDB. (s. f.-e). *Unique indexes*. MongoDB Manual. https://www.mongodb.com/docs/manual/core/index-unique/
- Mongoose. (s. f.-a). *Model.findOneAndUpdate()*. https://mongoosejs.com/docs/api/model.html
- Mongoose. (s. f.-b). *Schemas*. https://mongoosejs.com/docs/guide.html
- Mongoose. (s. f.-c). *Validation*. https://mongoosejs.com/docs/validation.html

- Node.js. (s. f.). *Node.js documentation*. https://nodejs.org/docs/latest/api/
- npm. (s. f.). *Scripts*. npm Docs. https://docs.npmjs.com/cli/v10/using-npm/scripts
- Preston-Werner, T. (s. f.). *Semantic versioning 2.0.0*. https://semver.org/
- React. (s. f.). *React*. https://react.dev/
- Robles, V. (2026). *Master en APIs RESTful con Node.js. Udemy*. https://www.udemy.com/course/master-en-apis-restful-con-nodejs-crea-3-backends-completos/
- Wiggins, A. (2011). *The twelve-factor app: III. Config*. https://12factor.net/config

8 Anexo: uso de IA

Durante el desarrollo del proyecto se utilizaron herramientas de inteligencia artificial como apoyo a la programación y a la redacción del informe. Las decisiones de diseño, la revisión del código y la responsabilidad sobre el contenido final son del equipo. Todo el código entregado fue revisado y probado antes de incorporarse al repositorio.

8.1. Herramientas utilizadas

Tabla 8. Herramientas utilizadas como apoyo durante el proyecto

Herramienta	Modelo o versión	Uso en el proyecto
Claude Code	gpt-oss:20b vía API de Ollama	Adaptación de la API base, refactorización a capas, implementación de recursos, validación y manejo de errores.
Claude (chat)	Sonnet 5.5	Redacción y estructuración del informe, revisión de criterios de la rúbrica, ejemplos de carga útil y diagramas.
Postman	–	Verificación funcional de endpoints; no es una herramienta de IA.
MongoDB Compass	–	Verificación de persistencia de datos; no es una herramienta de IA.

8.2. Uso por fase del proyecto

Tabla 9. Participación de la IA y del equipo por fase

Fase	Qué hizo la IA	Qué hizo el equipo	Verificación
Contrato y modelo de datos	Propuso borradores de la tabla de API y de esquemas.	Decidió recursos, nombres, sobre de respuesta con `status` y `statusCode`, y confirmación manual de entregas.	Revisión contra Hito 1 y rúbrica.
Capas y refactor	Movió lógica de controllers a services y creó repositories.	Definió reglas por capa y revisó los cambios.	Revisión de dependencias y recorrido de petición.
Endpoints y reglas de negocio	Generó una implementación base de recursos.	Corrigió errores y ajustó códigos HTTP y transiciones.	Postman y MongoDB Compass.
Errores y seguridad	Propuso manejo central de errores y gestión de credenciales.	Validó variables de entorno, `.gitignore` y control por rol.	Casos 400, 401, 403, 404 y 409 en Postman.
Informe	Redactó borradores de secciones.	Editó los textos y comprobó su correspondencia con el código.	Lectura cruzada con el repositorio.

8.3. Verificación de resultados

- **Postman:** colección con peticiones de éxito y error por recurso, verificando código de estado y cuerpo de respuesta.
- **MongoDB Compass:** después de las operaciones de escritura se comprobó la persistencia de documentos, sus estructuras y estados; también se revisaron colecciones e índices.
- **Revisión de código:** lectura de cada cambio generado antes de incorporarlo al repositorio.

8.4. Uso de la API base previa

El backend de Robin toma como punto de partida una API REST de una red social, desarrollada previamente como parte del curso de Udemy de Víctor Robles (Robles, 2026) *Master en APIs RESTful* (repositorio `CalquinL/API_Red_social`). Sobre esa implementación inicial se realizaron modificaciones para adecuarla a los requerimientos funcionales y a la arquitectura definida para Robin. Se reutilizaron algunos componentes como base de trabajo, mientras que otros fueron reescritos y se incorporaron funcionalidades específicas del proyecto, según se detalla en la siguiente tabla:

Tabla 10. Elementos reutilizados, reescritos y nuevos

Categoría	Elementos
Reutilizado	Autenticación JWT, modelos de usuario y publicación, paginación.
Reescrito	Estructura de capas, rutas por recurso y versionado, manejo de errores y formato de respuesta.
Nuevo	Necesidades, reservas, notificaciones, moderación de publicaciones y script de creación de la base de datos.